```python
print("Name: CHIRAG AGARWAL")
print("USN: 1WA24CS098")
print()

import math

path_cost = 0

# Initialize the board with the given stage from the image
board = ["O", " ", "X",
         "X", " ", " ",
         "X", "O", "O"]


def print_board(b):
    print(f"\n {b[0]} | {b[1]} | {b[2]} ")
    print("---|---|---")
    print(f" {b[3]} | {b[4]} | {b[5]} ")
    print("---|---|---")
    print(f" {b[6]} | {b[7]} | {b[8]} \n")


def is_winner(b, letter):
    # Check rows, columns, and diagonals
    winning_conditions = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8),  # Rows
        (0, 3, 6), (1, 4, 7), (2, 5, 8),  # Columns
        (0, 4, 8), (2, 4, 6)               # Diagonals
    ]

    return any(
        b[i] == b[j] == b[k] == letter
        for i, j, k in winning_conditions
    )


def is_board_full(b):
    return " " not in b
```

```
def get_empty_cells(b):
    return [i for i, spot in enumerate(b) if spot == " "]


# Minimax algorithm for unbeatable AI
def minimax(b, is_maximizing):

    if is_winner(b, "O"):
        return 1

    if is_winner(b, "X"):
        return -1

    if is_board_full(b):
        return 0

    if is_maximizing:
        best_score = -math.inf

        for cell in get_empty_cells(b):
            b[cell] = "O"

            score = minimax(b, False)

            b[cell] = " "

            best_score = max(score, best_score)

        return best_score

    else:
        best_score = math.inf

        for cell in get_empty_cells(b):
            b[cell] = "X"

            score = minimax(b, True)

            b[cell] = " "
```

```
            best_score = min(score, best_score)

        return best_score


def ai_move(b):
    global path_cost

    best_score = -math.inf
    move = None

    for cell in get_empty_cells(b):
        b[cell] = "O"

        score = minimax(b, False)

        b[cell] = " "

        if score > best_score:
            best_score = score
            move = cell

    if move is not None:
        b[move] = "O"

        # Increase path cost for AI move
        path_cost += 1


def player_move(b):
    global path_cost

    while True:
        try:
            choice = int(input("Choose a position (0-8): "))

            if b[choice] == " ":
                b[choice] = "X"

                # Increase path cost for player's move
```

```
                path_cost += 1

                break

            else:
                print("That spot is taken! Try again.")

        except (ValueError, IndexError):
            print("Invalid input. Enter a number between 0 and 8.")


# Main Game Loop
print("Welcome to Tic-Tac-Toe!")
print("You are X and the AI is O.")
print_board(board)

while True:

    # Player's turn
    player_move(board)
    print_board(board)

    # Check if player wins
    if is_winner(board, "X"):
        print("You win!")
        print("Path Cost:", path_cost)
        break

    # Check for draw
    if is_board_full(board):
        print("It's a tie!")
        print("Path Cost:", path_cost)
        break

    # AI's turn
    print("AI's turn...")
    ai_move(board)
    print_board(board)

    # Check if AI wins
```

```
    if is_winner(board, "O"):
        print("AI wins!")
        print("Path Cost:", path_cost)
        break

    # Check for draw
    if is_board_full(board):
        print("It's a tie!")
        print("Path Cost:", path_cost)
        break
```

Output:

```
Name: CHIRAG AGARWAL
USN: 1WA24CS098

Welcome to Tic-Tac-Toe!
You are X and the AI is O.

 O |   | X
---|---|---
 X |   |
---|---|---
 X | O | O

Choose a position (0-8): 5

 O |   | X
---|---|---
 X |   | X
---|---|---
 X | O | O

AI's turn...

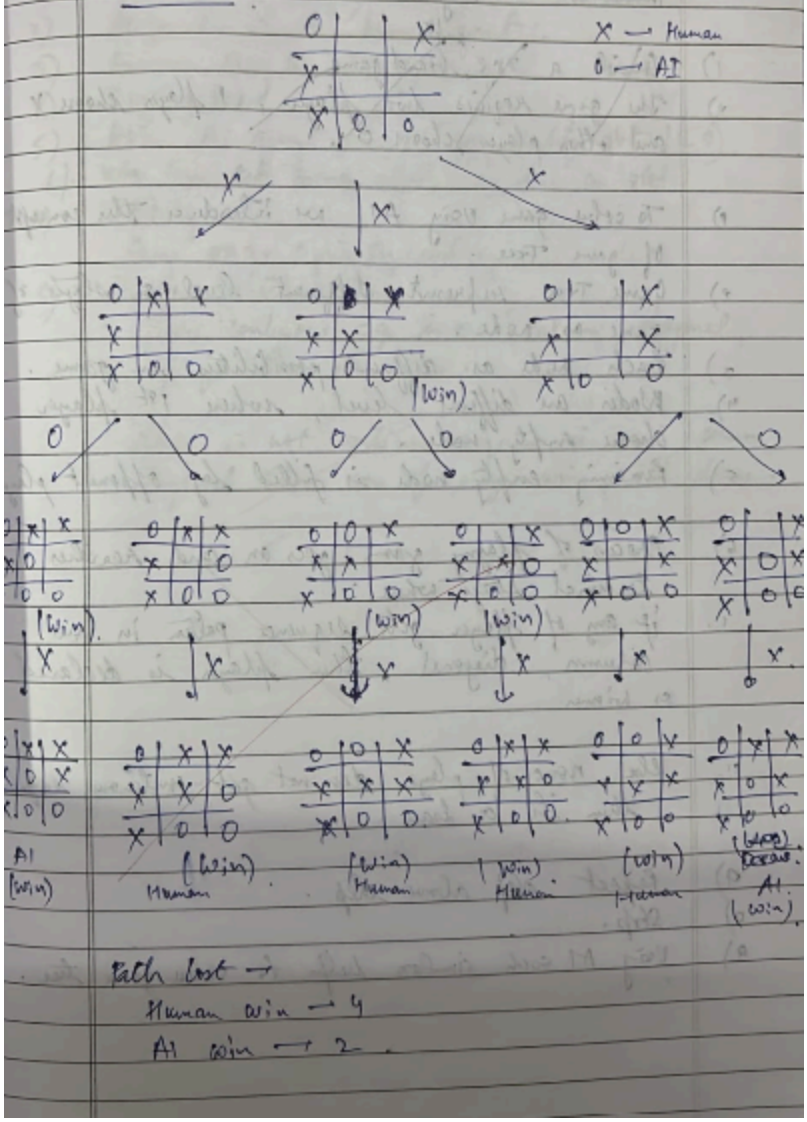
 O |   | X
---|---|---
 X | O | X
---|---|---
 X | O | O

AI wins!
Path Cost: 2
```

Date: 25/09/26

AI Lab

Tic Tac Toe



Path lost →

Human win → 4

AI win → 2

~~Pseudo code~~ Algorithm :-

1) Consider a 3x3 board game.
2) The game requires two player, 1 player choose X and other player choose O.

1) To solve game using AI we introduces the concept of game Tree.
2) Game Tree represent different levels or steps of game as node.
3) Each node as different possibilities in game.
4) Nodes are different level, where 1st player choose empty node.
5) Remaining empty node is filled by opponent player.
6) Process of playing game goes on and reaches terminal stage where :
7) if any of player gets sequence pattern in row, column, diagonal, then player is declared as winner.
8) Else none of player does not get continuous sequence then it is a draw.
9) Repeat ~~step~~ above step.
10) Stop.
11) Using AI such simulation helps to solve game like

1) Consider a 3X3 empty board game.
2) Assign an X to human agent.
3) Assign O to AI agent.
4) Display board.
5) Ask AI agent to fill an empty slot (1-9)
6) Then Turn ask human agent to fill an slot.
7) Similarly,
   Same process again and again.
8) Check whether if
   there continuous eg in row, column, diagonal,
   if yes then play wins.

   there is not continuous sequence in row, column,
   diagonal then non of player wins, it's draw.

Pseudocode →

Start the game.

Initialize 3x3 empty board.

Display board.

Human Turn →

Ask the user to enter row & column (1-3).

Check whether cell is empty

if yes, fill X in empty cell.

else ask to fill in diff cell

Display board

Check ~~same~~ for whether human won

if yes, stop

else continue.

Check if board is full

if yes, game is ~~a draw~~ winner, draw.

else continue

AI Turn :

Insert O in empty cell

Ask to fill empty cell

Check empty cell

if yes, fill O in empty cell

else ask to fill in diffent cell.

Display board.

Check whether AI wons.

if yes, stop declar AI as winner.

else continue

check if board is full

if yes, now is winner, draw.

else continue.

Date:

Output →

Name: Uvraj Agarwal
USN: 1NA24CS098

Welcome To Tic-Tac-Toe!
You are X and AI is O.

O		X
X		
X	O	O

i) Choose a position (0-8): 1

6	X	X
X		
X	O	O

AI Turn.

O	X	X
X	O	
X	O	O

AI wins!
Path cost: 2

ii) Choose a position (0-8): 4

O		X
X	X	
X	O	O

You wins!
Path cost: 1

ii. Choose a partition (0-8) : 5

0		X
X		X
X	0	0

AI turn

0		X
X	0	X
X	0	0

AI wins!

Path cost : 2